

Arrays & Structs in Memory

```
int arr[3]
a[0] = -1
⋮
```

arr	-1	arr[0]
	5	arr[1]
	3	arr[2]

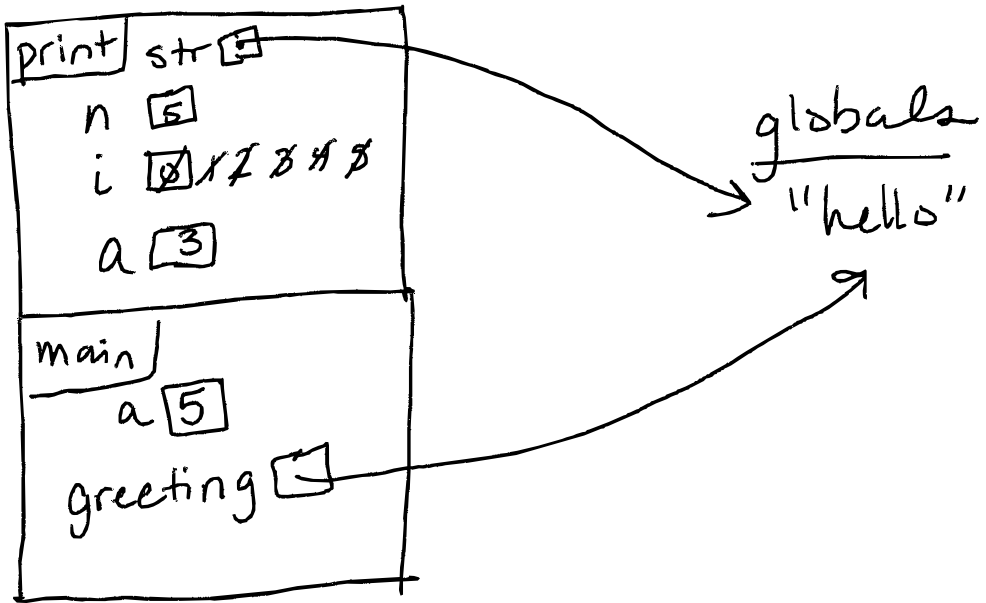
- contiguous
- 4 bytes apart (ints)

```
Struct dataT data;
data.v[0] = 2;
⋮
```

```
data.c = 'g';
data.y = 3.14;
```

data	2	v[0]
	-3	v[1]
	0	v[2]
	'g'	c
	xxxx	padding
	3.14	y

Stack Diagram Practice

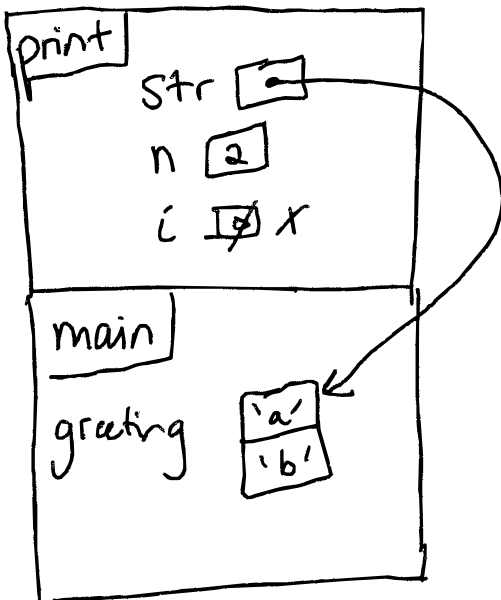


output: h-e-l-l-o-

Stack Diagram

Practice

- * not a true string (no '\0' at the end)
- * const means we will not change str inside print



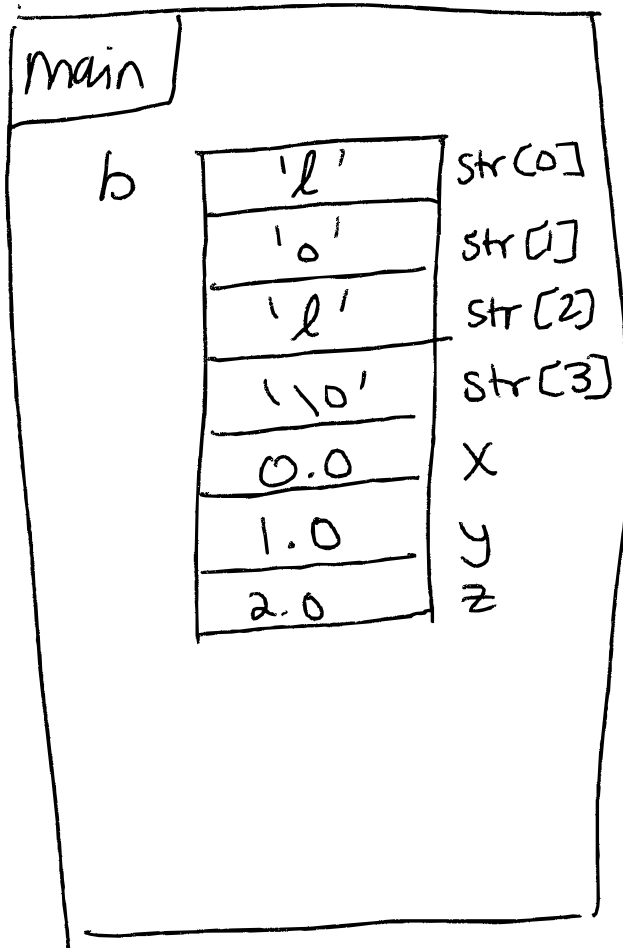
How big is 'b' in
memory?

char str[4] $\rightarrow$ 4 bytes

float x, y, z $\rightarrow$ $3 \cdot 4$ bytes
 $= 12$ bytes

so at least 16 bytes

Draw 'b' in memory



Pointer or not?

char test[3] ✓

char* test ✓

char test x

const char* test ✓

char** test ✓

char* test[5] ✓

char test[] ✓

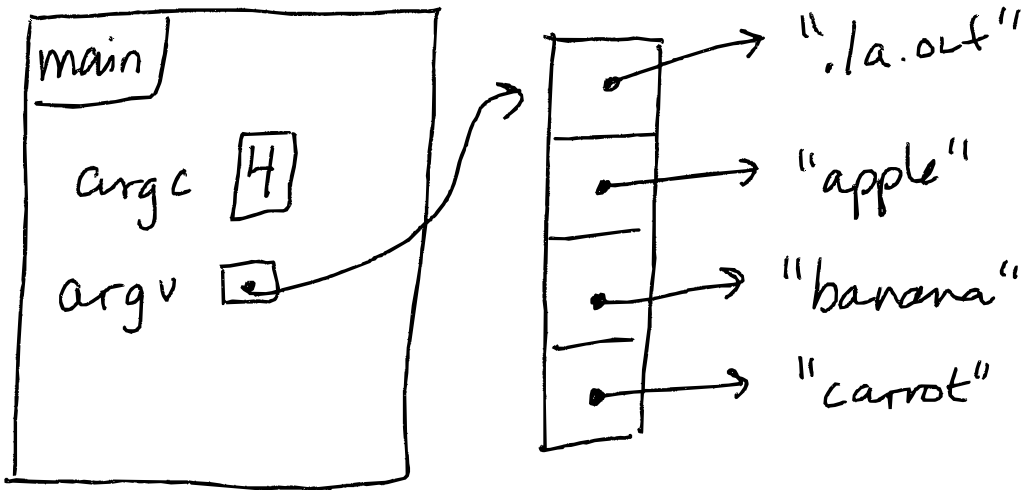
int test x

int test[1000] ✓

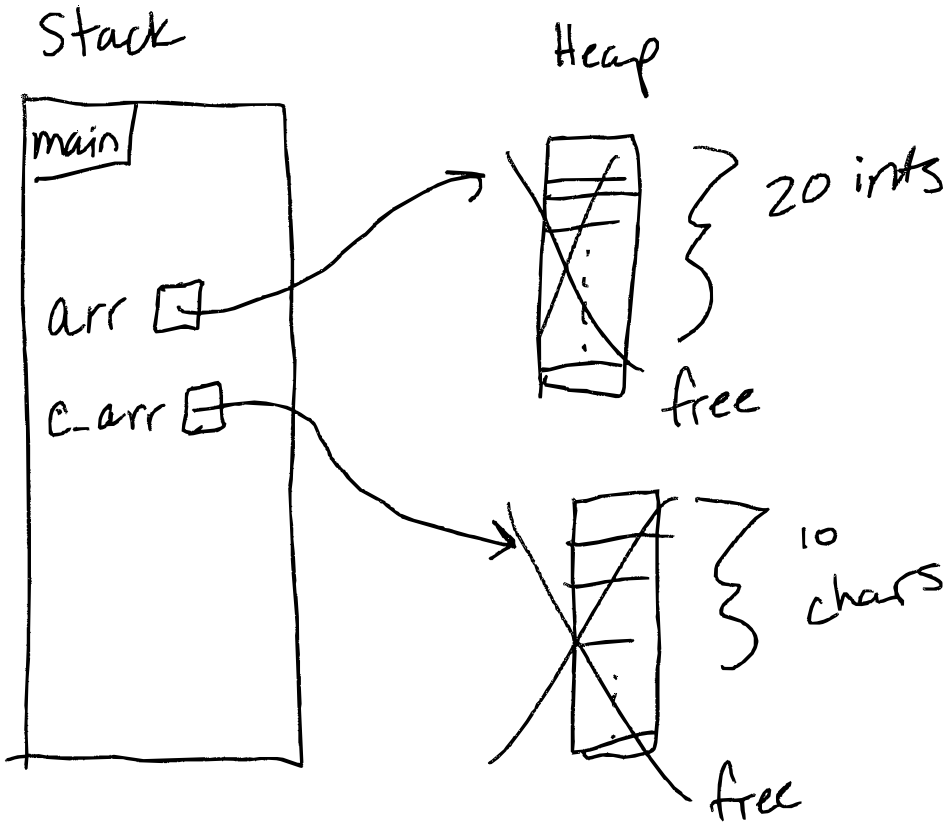
int* test ✓

Command Line Args

`char** argv` → array of strings

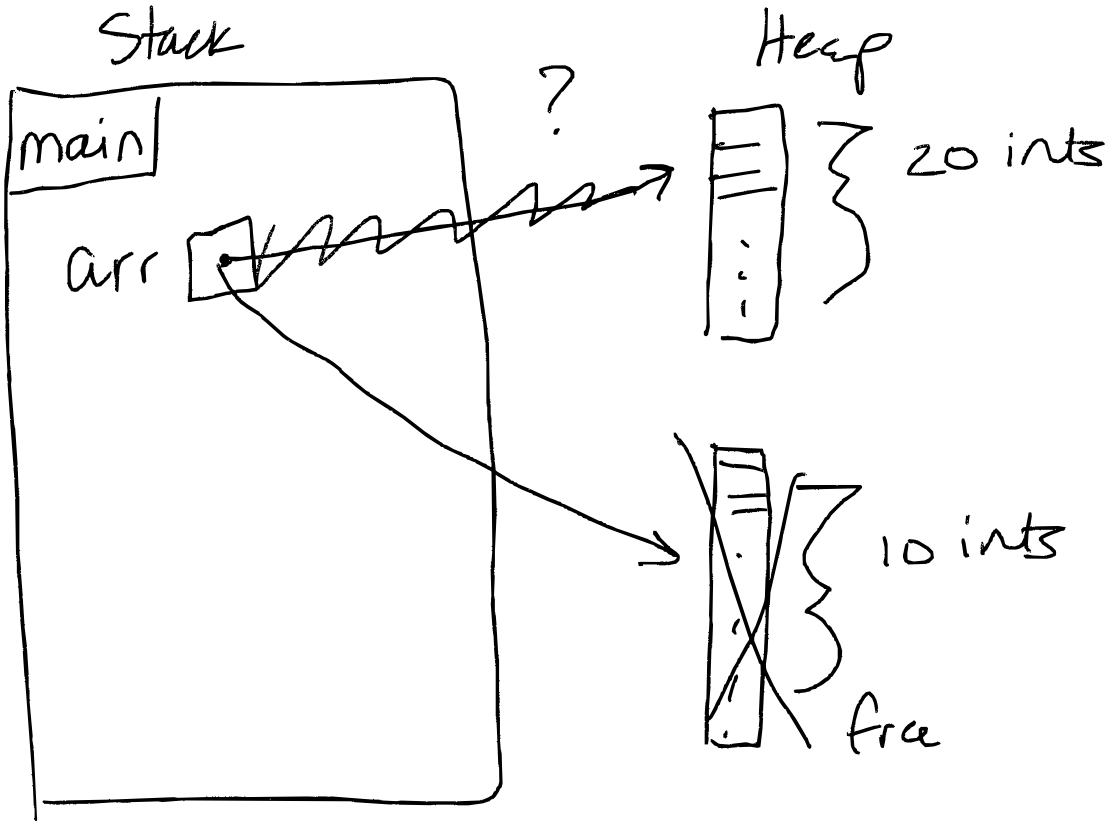


Malloc & Free (slide 35)



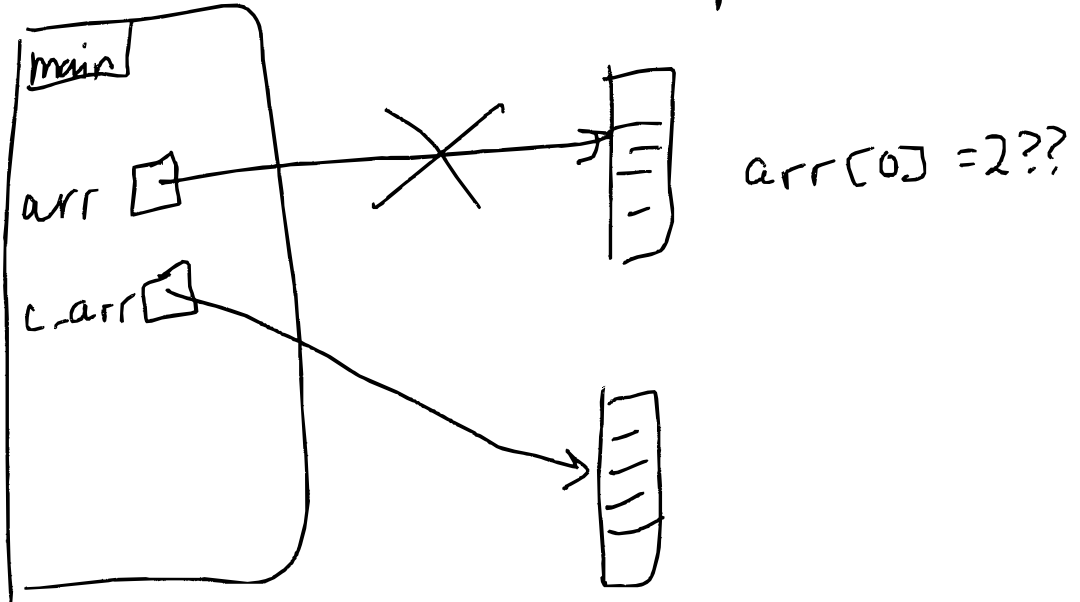
Malloc & Free

Memory Leak

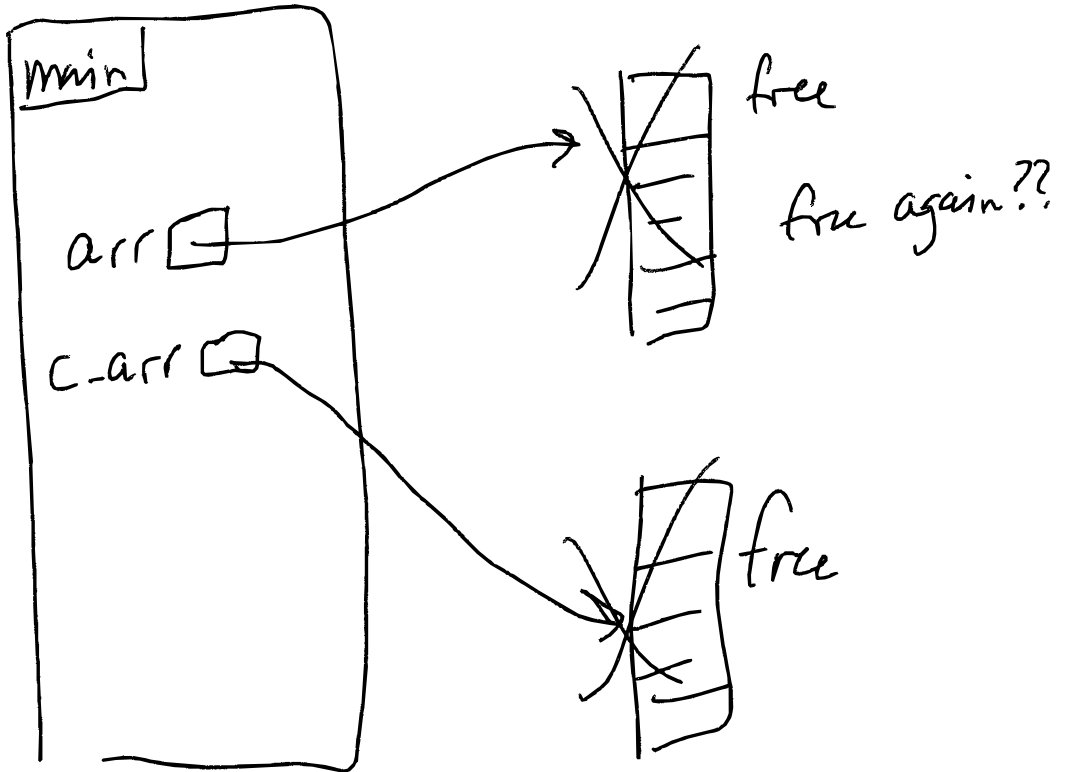


Malloc & Free : use after free

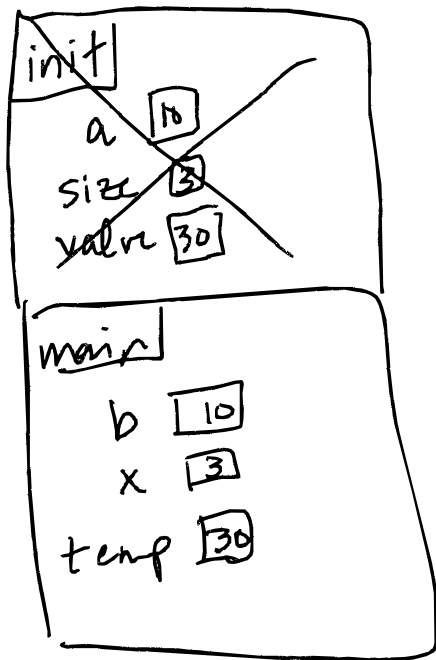
Heap



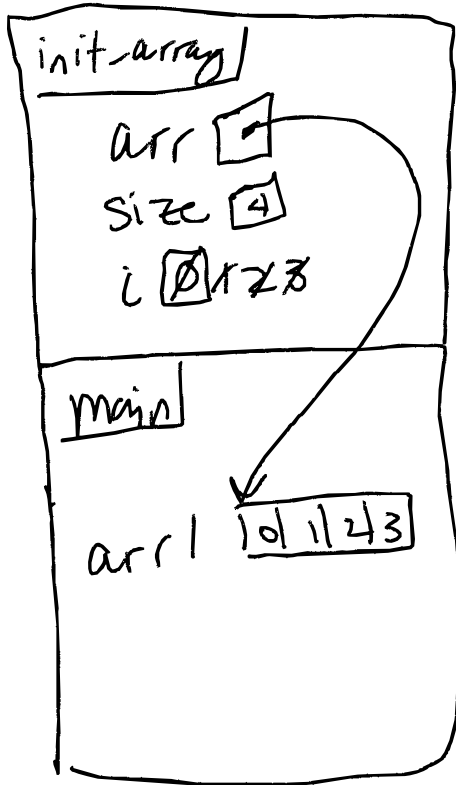
Malloc & free : double free



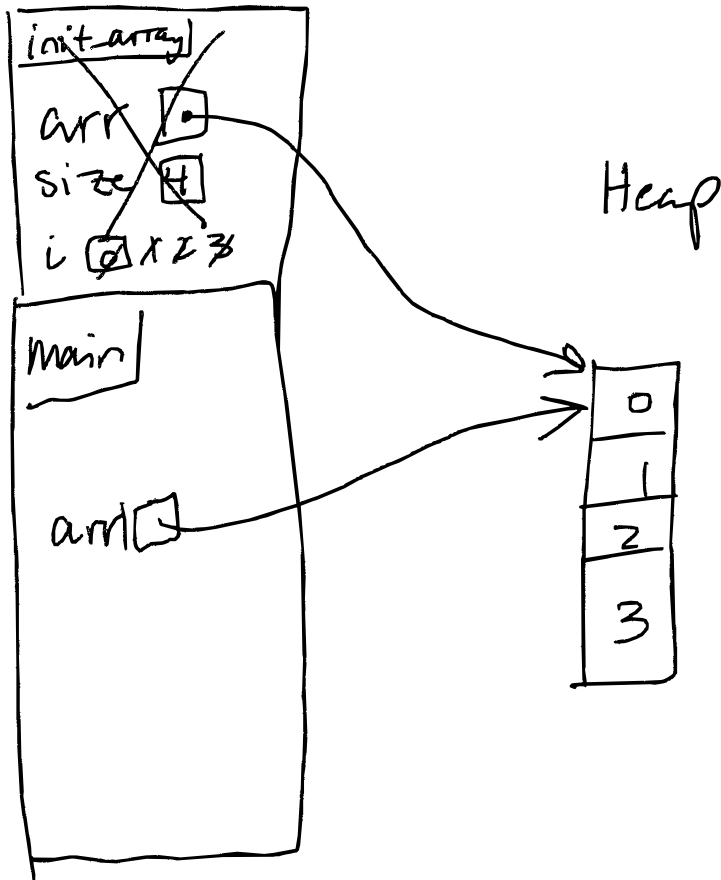
Passing Simple Types to functions



Passing arrays to functions



Stack & Heap: Same
example as last time,
but let's malloc the array
instead.



Reverse Array

int left = 0 right = n - 1
 = 5 - 1 = 4

while (left < right)

// do swap

left++; move to
right--; middle

[1, 2, 3, 4, 5]

temp = 1



arr[0] = 5

[5, 2, 3, 4, 1]

arr[4] = 1

2nd round:

left = 1

right = 3

temp = arr[1] = 2

arr[1] = arr[3] = 4

arr[3] = temp = 2

[5, 4, 3, 2, 1]